Seguridad en Sistemas Embebidos

Amenazas, criptografía aplicada y buenas prácticas para dispositivos embebidos e IoT

OCT 2026

Agenda

01 Contexto y por qué importa

02 Tipos de ataques a sistemas embebidos

03 Atributos de seguridad

04 Diseño seguro y proceso de desarrollo

05 Criptografía aplicada (hash, cifrado, firmas)

06 Secure Boot y protección de firmware

07 Hardware de seguridad y librerías

08 Buenas prácticas y recursos

Contexto: Sistemas embebidos, CPS e IoT

Los sistemas embebidos ya no están aislados. Forman parte de sistemas ciberfísicos (CPS) e Internet de las Cosas (IoT) que controlan infraestructura crítica, industria, ciudades y hogares.

Industrial IoT

SCADA, PLCs, robots, líneas de producción

Smart Cities

Semáforos, iluminación, sensores urbanos

Smart Grid

Medidores, subestaciones, generación distribuida

Automoción

ECUs, ADAS, vehículos conectados

Salud

Dispositivos médicos, monitores, bombas de infusión

Hogar / Consumer

Cámaras, routers, electrodomésticos, wearables

Por qué importa la seguridad

Ejemplos reales que marcaron la agenda

2010

Stuxnet

Gusano dirigido a PLCs Siemens. Dañó centrifugadoras de enriquecimiento de uranio en Irán. Primer malware conocido contra infraestructura industrial.

2015

Jeep Cherokee

Investigadores tomaron control remoto de un vehículo a través de la red celular (Uconnect). Frenos, dirección y motor comprometidos.

2016

Mirai

Botnet de cámaras y routers IoT. Lanzó DDoS masivo (Dyn). Demostró el riesgo de credenciales por defecto y dispositivos no parcheados.

2021+

Supply-chain y OT

Ataques a cadena de suministro (firmware, librerías) y ransomware en entornos OT/ICS. Impacto en disponibilidad y seguridad física.

Desafíos específicos de los sistemas embebidos

Recursos limitados

CPU, memoria y energía restringidas. Criptografía pesada es costosa.

Larga vida útil

Dispositivos en campo 10–20 años. Actualizaciones difíciles o imposibles.

Acceso físico

Muchos dispositivos están expuestos. Atacante puede tocar el hardware.

Heterogeneidad

Múltiples arquitecturas, RTOS, sin MMU, sin SO estándar.

Cadena de suministro

Firmware y componentes de terceros. Riesgo de backdoors o clones.

Costo vs seguridad

Time-to-market y costo unitario suelen priorizarse sobre seguridad.

Tipos de ataques

Clasificación práctica orientada a sistemas embebidos

1. **Ataques de software**
   Explotación de vulnerabilidades, clonado, DoS, RCE
2. **Canal lateral (Side-Channel)**
   Consumo de energía, EM, timing
3. **Inyección de fallos**
   Voltage/clock glitching, láser, EM
4. **Ataques físicos**
   Microprobing, decapsulación, lectura de memoria
5. **Cadena de suministro**
   Componentes maliciosos, firmware alterado, spoofing

1. Ataques de software

Explotan vulnerabilidades en el código, protocolos o algoritmos de seguridad.

Clonado de dispositivos

Copia funcional del firmware y datos de seguridad (claves, certificados).

Ruptura de protección contra copia

Anulación de ofuscación, cifrado de firmware o anti-debug.

Denegación de servicio (DoS)

Sobrecarga de red, memoria o CPU hasta dejar el dispositivo inoperable.

Explotación de vulnerabilidades

Buffer overflow, format string, use-after-free → ejecución de código arbitrario.

Credenciales por defecto / hardcodeadas

Una de las causas más frecuentes de compromiso masivo (Mirai y similares).

2. Ataques de canal lateral (Side-Channel)

Analizan señales físicas emitidas durante la operación normal para inferir secretos (claves, datos).

Análisis de consumo de energía (SPA/DPA)

Fluctuaciones de corriente revelan operaciones internas y bits de clave.

Emisiones electromagnéticas

Radiación EM del chip permite reconstruir operaciones criptográficas.

Análisis de tiempo (Timing)

Variaciones en el tiempo de ejecución dependen de los datos o la clave.

Análisis de cache / microarquitectura

En dispositivos más potentes (Linux embebido, SoCs) también aplica.

3. Inyección de fallos (Fault Injection)

Se induce un error controlado para que el sistema se desvíe (salte verificaciones, filtre datos, etc.).

Voltage Glitching

Interrupción breve del suministro de tensión para causar errores transitorios en la CPU.

Clock Glitching

Alteración de la señal de reloj para que instrucciones se ejecuten de forma incorrecta u omitan pasos.

Inyección láser / EM

Láser de precisión o pulsos electromagnéticos sobre zonas específicas del die.

Objetivo típico

Saltar chequeos de firma de firmware, desactivar protecciones de depuración, extraer claves.

4. Ataques físicos e intrusivos

Requieren acceso directo al hardware del dispositivo.

Microprobing

Sondas en pistas internas del chip para leer buses o memoria.

Decapsulación y análisis del die

Retiro de la encapsulación para estudiar el layout o leer memoria no volátil.

Lectura de memoria

Extracción de firmware desde Flash/EEPROM mediante interfaces de depuración o ataques combinados.

Tampering de sensores

Manipulación de entradas físicas (temperatura, voltaje) para inducir comportamientos inseguros.

5. Cadena de suministro y otros

Ataques a la cadena de suministro

Introducción de componentes o firmware maliciosos en fabricación, distribución o actualizaciones OTA.

Spoofing / suplantación

Dispositivo malicioso se hace pasar por un nodo legítimo de la red o del sistema.

Firmware no firmado o mal verificado

Actualizaciones que permiten instalar código arbitrario sin autenticación fuerte.

Dependencias de terceros

Librerías y SDKs con vulnerabilidades (o backdoors) que se propagan a productos finales.

Atributos de seguridad

Confidencialidad	Solo entidades autorizadas pueden leer la información. → Cifrado
Integridad	Los datos no han sido alterados (intencional o accidentalmente). → Hash, MAC, firmas
Autenticidad	Garantía del origen de los datos o de la identidad de la entidad. → HMAC, firmas, certificados
No repudio	Imposibilidad de negar la autoría de un mensaje o acción. → Firmas digitales + PKI
Disponibilidad	El sistema permanece operativo ante ataques o fallos. → Diseño resiliente, mitigación de DoS

Diseño seguro desde el inicio

La seguridad no se «añade» al final. Debe tratarse como requisito funcional desde el diseño.

Secure by Design

Amenazas modeladas (STRIDE, attack trees) en la fase de requisitos.

Mínimo privilegio

Cada componente solo tiene los permisos estrictamente necesarios.

Defensa en profundidad

Múltiples capas (hardware, boot, OS, aplicación, red).

Actualizabilidad

Canal seguro de actualización de firmware (firmado + verificado).

Fail-secure

Ante fallo o ataque, el sistema degrada a un estado seguro, no abierto.

Auditoría y trazabilidad

Logs, SBOM (Software Bill of Materials), gestión de vulnerabilidades.

Atributos de seguridad

Confidencialidad,
integridad,
autenticidad,
no-repudio,
disponibilidad.

Integridad: funciones hash

Una función hash criptográfica mapea datos de longitud variable a un digesto de longitud fija.

Propiedades deseables

- Irreversible (preimagen difícil)
- Rápida de calcular
- Efecto avalancha (1 bit → hash muy distinto)
- Determinista (mismo input → mismo hash)
- Salida de longitud fija
- Resistente a colisiones

Recomendación actual

Usar SHA-256 o SHA-3 (SHA-3-256 / SHAKE). Evitar MD5 y SHA-1 para cualquier uso de seguridad (están rotos o debilitados). CRC32 solo sirve para detección de errores, no para integridad criptográfica.

Confidencialidad: cifrado simétrico

Misma clave para cifrar y descifrar. Rápido y eficiente; ideal para grandes volúmenes de datos.

AES

Estándar de facto. Claves de 128, 192 o 256 bits. Preferir AES-128 o AES-256.

Modos recomendados

GCM o CCM (autenticados). Evitar ECB. CBC requiere IV aleatorio y padding correcto.

Ventajas

Alto rendimiento, implementable en hardware, bajo consumo.

Desafío principal

Distribución y gestión segura de la clave compartida.

Autenticación simétrica: HMAC

Hash-based Message Authentication Code. Combina una función hash con una clave secreta.

Qué aporta

Integridad + autenticación del origen (quien conoce la clave puede verificar).

Qué no aporta

No cifra el mensaje. No proporciona no-repudio (ambas partes conocen la clave).

Uso típico

APIs (AWS Signature), tokens, verificación de mensajes en protocolos embebidos.

Recomendación

HMAC-SHA256. Clave de al menos 128 bits, preferiblemente 256.

Cifrado asimétrico y firmas digitales

Par de claves: pública (distribuible) y privada (secreta). Base de firmas, PKI y no-repudio.

RSA

Clásico y ampliamente soportado. Claves largas (2048+). Más lento y costoso en embebidos.

Curvas elípticas (ECC)

ECDSA / Ed25519. Claves mucho más cortas con seguridad equivalente. Ideal para embebidos.

Firma digital

Integridad + autenticidad + no-repudio. Se firma el hash del mensaje con la clave privada.

Limitación

Lento y con límite de tamaño. Para datos grandes se combina con cifrado simétrico (esquema híbrido).

Esquema híbrido (puente simétrico)

La forma práctica de cifrar datos grandes de forma segura:

1. Generar una clave simétrica aleatoria (sesión).

2. Cifrar los datos con AES (o similar) usando esa clave.

3. Cifrar la clave de sesión con la clave pública del destinatario (RSA/ECC).

4. Enviar: datos cifrados + clave de sesión cifrada.

5. El destinatario descifra la clave de sesión con su privada y luego los datos.

Este es el patrón usado en TLS, PGP, actualizaciones de firmware cifradas, etc.

Secure Boot / Firmware firmado

Garantiza que solo se ejecuta código auténtico e íntegro desde el arranque.

En fabricación / desarrollo

Se genera par de claves. El firmware se hashea y se firma con la clave privada. La pública se graba de forma inmutable en el dispositivo.

En cada arranque

El bootloader calcula el hash del firmware, verifica la firma con la clave pública. Si es válida → arranca. Si no → se detiene o entra en modo seguro.

Actualizaciones OTA

El nuevo firmware también debe venir firmado. El dispositivo verifica antes de escribir y de ejecutar.

Hardware de seguridad

Características que ayudan a elevar el nivel de protección:

Raíz de Confianza (Hardware Root of Trust – RoT) y Secure Boot Loader

Bloque inmutable (usualmente ROM de arranque protegida contra escritura y eFuses OTP) que valida la primera etapa del software antes de transferir el control

TRNG

Un generador de números aleatorios por hardware no se basa en algoritmos deterministas (como un PRNG o un generador pseudoaleatorio en software), sino en la entropía física del silicio.

Tamper detection

Sensores de voltaje, temperatura, malla de seguridad del die.shields metálicos contra lectura óptica, blindaje contra análisis diferencial de potencia (DPA) y filtros contra ataques de inyección de fallos (glitch attacks en reloj o alimentación).

Aceleradores criptográficos

AES, SHA, ECC/RSA en hardware (velocidad y resistencia a side-channel).

Almacenamiento seguro de claves

Almacenamiento seguro de claves donde la clave raíz nunca es leída por la CPU en texto plano; el hardware realiza el descifrado o la firma directamente dentro del enclave criptográfico.

TrustZone / Secure Element / TEE

Divide el espacio de memoria, periféricos e interrupciones en dominios seguro y no-seguro (ej. Arm TrustZone para Cortex-M23/M33/M55/M85, o Physical Memory Protection - PMP en núcleos RISC-V).

Algunos microcontroladores

Silicon Labs EFR32xG24 / Series 2 (Secure Vault High)
Subsistema de seguridad aislado por hardware (Secure Vault), SRAM-PUF integrada, contramedidas avanzadas DPA y detecto
de manipulación física (tampering).

NXP LPC55S69 / MCX N Series
Arm TrustZone (Cortex-M33 dual), motor EdgeLock / CASPER, PUF física para generación de claves efímeras sin guardado en
Flash, y aceleradores criptográficos blindados.

STMicroelectronics STM32H5 / STM32U5
Arm TrustZone, aislamiento de memoria física (HDP/WRP), acelerador criptográfico PKA/AES aislado, Secure Boot / Secure
Firmware Install (SFI) de fábrica.

Infineon AURIX TC3xx / TC4x (Automoción / Crítico)
Módulo HSM (Hardware Security Module) dedicado con núcleo propio independiente del CPU principal, lockstep cores, y memo
segura cifrada.
Common Criteria EAL4+ / EAL5+ (en perfiles HSM), ISO 26262 ASIL-D.

ESP32
Aceleradores por hardware para AES, SHA y RSA/ECC, un generador TRNG basado en ruido térmico RF, cifrado transparente
memoria Flash externa y Secure Boot basado en firmas digitales, anclando sus claves criptográficas de forma irreversible en u
bloque de eFuses protegidos contra lectura.

Librerías recomendadas

mbedTLS / PSA Crypto	Amplia, bien mantenida, orientada a embebidos. Base de muchos stacks TLS.
wolfSSL	Ligera, certificable, buena para dispositivos con recursos muy limitados.
BearSSL	Diseñada para ser pequeña y resistente a side-channel. Ideal para microcontroladores.
micro-ecc	ECC puro en C, sin dependencias, resistente a side-channel conocidos. Muy compacta.
TinyCrypt	Conjunto mínimo de primitivas (AES, HMAC, ECC) de Intel para IoT.
libsodium / TweetNaCl	API simple y segura (cuando hay más recursos).

Librerías certificadas

wolfSSL (wolfCrypt):

Es la más certificada del grupo. Su motor criptográfico subyacente (wolfCrypt) cuenta con validación FIPS 140-2 / FIPS 140-3 (Nivel 1 y Nivel 2) otorgada por el NIST/CMVP.

Certificaciones adicionales para entornos críticos: DO-178C DAL A (aviónica), ISO 26262 (automoción) e IEC 61508 (seguridad funcional industrial).

wolfSSL

La API y arquitectura de PSA Certified (impulsada por Arm y mantenida dentro del ecosistema Mbed TLS / TrustedFirmware) cuenta con certificaciones formales en silicio y software de evaluación de nivel PSA Certified Level 1, Level 2 y Level 3.

Ha recibido validaciones de algoritmos bajo el programa CAVP (Cryptographic Algorithm Validation Program) del NIST, aunque históricamente no busca FIPS 140 completo de forma nativa a menos que se integre con enclaves certificados.

Buenas prácticas y consejos finales

- Evaluar el impacto real de un ataque exitoso antes de decidir el nivel de seguridad.
- Incluir requisitos de seguridad desde el inicio del diseño (no como «parche»).
- No inventar algoritmos criptográficos ni reimplementar los conocidos «a mano».
- Usar librerías auditadas y mantenidas. Preferir código abierto cuando sea posible.
- Cada dispositivo debería tener su propia clave privada (no claves compartidas de fábrica).
- Evitar guardar claves en memoria no volátil de forma accesible. Preferir generación en dispositivo o inyección segura.
- Implementar Secure Boot + actualizaciones firmadas siempre que el hardware lo permita.
- Proteger interfaces de depuración (JTAG/SWD) en producción.
- Mantener un inventario de software (SBOM) y un proceso de respuesta a vulnerabilidades.

Dónde profundizar

NIST	SP 800-193 (Platform Firmware Resiliency), SP 800-90A (RNG), FIPS 140-3
OWASP	IoT Top 10, Embedded Application Security
PSA Certified / Arm	Platform Security Architecture – niveles de seguridad para dispositivos
IEC 62443	Seguridad para sistemas de control industrial (ICS/OT)
CISA / ENISA	Guías y alertas sobre amenazas a infraestructura e IoT
Papers clásicos	Trabajos de side-channel (Kocher et al.), fault injection, secure boot

Resumen rápido

1	**Amenazas reales**	Software, side-channel, fault injection, físico y supply-chain.
2	**Atributos**	Confidencialidad, integridad, autenticidad, no-repudio, disponibilidad.
3	**Criptografía**	SHA-2/3, AES-GCM, HMAC-SHA256, ECDSA/Ed25519 + esquema híbrido.
4	**Arranque seguro**	Firmware firmado + verificación en boot + actualizaciones autenticadas.
5	**Proceso**	Seguridad desde el diseño, librerías auditadas, claves únicas por dispositivo.

¿Preguntas?

Gracias

javier_jorge@disroot.org